



Linguagem de Programação: C++

Universidade Vila Velha

Comprometida com a excelência no ensino, pesquisa e inovação.

✉ wanderson.santana@uvv.br

Capítulo 2

Introdução a Classes, Objetos, Atributos e Funções-Membro

Resumo

Nesta unidade nosso objetivo é compreender a transição da programação estruturada para a Orientação a Objetos em C++, modelando entidades do mundo real por meio de encapsulamento, construtores e separação de interface e implementação.

“A simplicidade é a sofisticação suprema.”

— Leonardo da Vinci

2.1 Visão Geral da Programação Orientada a Objetos

A programação orientada a objetos (POO) constitui um paradigma de programação no qual dados e operações relacionadas a esses dados são organizados em entidades denominadas **objetos**. Em C++, esse paradigma é implementado por meio de recursos como classes, objetos, encapsulamento, herança, polimorfismo e composição.

A principal mudança conceitual em relação à programação estruturada não está simplesmente na utilização de uma nova sintaxe. Na programação estruturada, é comum organizar o programa a partir de funções que operam sobre dados. Na orientação a objetos, dados e operações que possuem uma relação conceitual são agrupados em uma mesma entidade, permitindo representar no programa elementos relevantes do domínio do problema.

Essa abordagem torna possível construir abstrações que aproximam a estrutura do software das entidades e relações presentes no problema que está sendo modelado.

2.1.1 Classes e Objetos: Moldes e Instâncias

Uma **classe** pode ser entendida como uma especificação que descreve as características e os comportamentos que os objetos de determinado tipo devem possuir. A classe define, portanto, quais dados serão armazenados e quais operações poderão ser realizadas sobre esses dados.

Um **objeto**, por sua vez, é uma instância concreta de uma classe. Enquanto a classe representa a descrição de uma categoria de entidades, o objeto representa uma entidade efetivamente criada durante a execução do programa.

Uma analogia útil é considerar uma planta arquitetônica. A planta descreve como uma determinada construção deve ser organizada, mas não constitui, por si só, uma casa física. A partir de uma mesma planta podem ser construídas diversas casas. De maneira semelhante, uma classe descreve a estrutura e os comportamentos de seus objetos, e diferentes objetos podem ser criados a partir dessa mesma classe.

Assim, podemos estabelecer a seguinte correspondência:

É importante não interpretar essa analogia literalmente. Uma classe não é simplesmente

Tabela 2.1: Relação conceitual entre classe e objeto.

Conceito	Programação Orientada a Objetos	Analogia
Classe	Especificação que define dados e comportamentos de uma categoria de objetos.	Planta ou molde de fabricação.
Objeto	Instância concreta de uma classe, com seu próprio estado.	Casa construída a partir da planta.
Atributo	Dado que representa uma característica ou estado do objeto.	Característica da casa, como área ou número de quartos.
Função-membro	Operação que define um comportamento associado ao objeto.	Ação que pode ser realizada sobre a entidade.

um “molde de memória”, assim como um objeto não é apenas um conjunto de variáveis. A classe também estabelece uma interface para as operações que podem ser realizadas sobre seus objetos e pode controlar como o estado interno desses objetos será acessado ou modificado.

2.1.2 A Analogia do Motorista, do Pedal e do Mecanismo Interno

Uma segunda analogia, particularmente útil para compreender o conceito de **encapsulamento**, é a relação entre um motorista e um automóvel.

O motorista não precisa conhecer todos os mecanismos internos responsáveis pelo funcionamento do veículo para conduzi-lo. Para acelerar, por exemplo, ele utiliza uma interface externa — o pedal do acelerador. Ao pressioná-lo, uma cadeia de mecanismos internos é acionada, mas os detalhes dessa implementação permanecem ocultos do motorista.

Podemos estabelecer uma correspondência conceitual:

Tabela 2.2: Analogia entre um automóvel e conceitos fundamentais da orientação a objetos.

Automóvel	Conceito em POO	Função conceitual
Automóvel	Objeto	Entidade concreta que possui estado e comportamento.
Projeto do automóvel	Classe	Especificação que define características e comportamentos.
Pedal do acelerador	Interface / função-membro	Meio pelo qual o usuário solicita determinada operação.
Motor e mecanismos internos	Implementação / estado interno	Detalhes necessários para realizar a operação, mas que podem ser ocultados do usuário.

Essa analogia introduz uma ideia central da orientação a objetos: **o código que utiliza um objeto não precisa necessariamente conhecer os detalhes internos de sua implementação**. Ele interage com uma interface bem definida.

No exemplo do automóvel, o motorista solicita uma ação por meio do pedal. Ele não precisa controlar diretamente a injeção de combustível, a abertura das válvulas ou o funcionamento do sistema de transmissão. De maneira análoga, uma função que utiliza um objeto deve, sempre que possível, interagir com ele por meio de operações públicas definidas pela própria classe.

Esse princípio será posteriormente formalizado por meio dos especificadores de acesso `public` e `private` e do conceito de **encapsulamento**.

2.1.3 Estado e Comportamento de um Objeto

Um **objeto** pode ser analisado sob duas perspectivas complementares: **estado** e **comportamento**.

O **estado** corresponde aos **dados** que representam a **situação atual** do objeto. Em C++, esses dados são normalmente representados por **membros de dados**, também denominados **atributos** em uma abordagem introdutória.

O **comportamento** corresponde às **operações** que podem ser **realizadas** pelo objeto. Em C++, essas operações são implementadas por meio de **funções-membro**.

Considere, por exemplo, uma **entidade** que represente uma conta bancária. Seu **estado** pode incluir o **nome** do titular e o **saldo** disponível. Seu **comportamento** pode incluir operações como **depositar**, **retirar** e **consultar** o saldo.

```
1 class ContaBanco {  
2     // Estado  
3     std::string nomeTitular;  
4     double saldo;  
5  
6     // Comportamento  
7     void depositar(double valor);  
8     void retirar(double valor);  
9 };
```

O exemplo anterior ainda não apresenta uma **classe completa** e será desenvolvido progressivamente nas próximas seções. Seu objetivo é apenas estabelecer a distinção fundamental:

- **Estado:** informações que descrevem a situação atual do objeto;
- **Comportamento:** operações que podem **consultar** ou **modificar** esse **estado**.

Essa separação conceitual será fundamental para compreender por que os **atributos** de uma classe normalmente não devem ser disponibilizados **diretamente** ao código **externo**.

👍 Boa Prática de Programação 2.1: Pense em termos de responsabilidades

Ao projetar uma **classe**, não pense apenas em quais **dados** precisam ser armazenados. Pergunte também quais **operações** pertencem naturalmente à entidade e quais **regras** devem ser **preservadas** pela própria classe. Essa abordagem favorece classes mais coesas e reduz a **dependência** entre diferentes partes do programa.

2.1.4 Da Programação Estruturada para a Orientação a Objetos

A transição para a orientação a objetos não significa abandonar funções, variáveis ou estruturas de controle. Esses elementos continuam existindo em C++. A mudança está principalmente na maneira como os **componentes** do programa são **organizados**.

Em uma abordagem **estruturada**, poderíamos representar uma conta bancária por meio de **variáveis e funções independentes**:

```
1 std::string nomeTitular;  
2 double saldo;  
3  
4 void depositar(double valor);  
5 void retirar(double valor);
```

Na orientação a objetos, os **dados** e as **operações** relacionados à conta podem ser reunidos em uma classe:

```
1 class ContaBanco {  
2     std::string nomeTitular;  
3     double saldo;  
4     void depositar(double valor);  
5     void retirar(double valor);  
6 };
```

A vantagem não está simplesmente em “colocar funções dentro de uma classe”. A **classe** passa a representar uma **abstração** do domínio e pode estabelecer regras sobre como seu **estado** é criado, consultado e modificado.

Esse mecanismo será desenvolvido nas próximas seções, começando pela criação de uma classe simples e pela instanciação de seus primeiros objetos.

2.2 Definindo uma Classe com uma Função-Membro

Depois de compreender a distinção entre `classes` e `objetos`, podemos construir uma primeira classe em C++. O objetivo desta seção não é apresentar todos os recursos disponíveis para uma classe, mas demonstrar, de forma progressiva, como uma classe é `declarada`, como uma `função-membro` é `definida` e como um `objeto` pode utilizar essa `função`.

Para isso, será utilizado um exemplo deliberadamente simples. A classe `ContaBanco` representará, inicialmente, uma conta bancária capaz de executar uma operação elementar. Os mecanismos de `encapsulamento`, `atributos privados` e `construtores` serão introduzidos posteriormente.

2.2.1 A Declaração de uma Classe

Uma classe é declarada utilizando a palavra-chave `class`, seguida pelo `nome` que identificará o novo tipo. O `conteúdo` da classe é delimitado por chaves e a declaração deve ser encerrada por `ponto e vírgula`.

A estrutura geral pode ser representada da seguinte maneira:

```
1 class NomeDaClasse {  
2     // Membros da classe  
3 };
```

O trecho anterior ainda não define nenhuma característica ou comportamento específico. Ele apenas declara um novo tipo denominado `NomeDaClasse`.

Podemos agora substituir o nome genérico por uma classe que represente uma `entidade` do domínio do nosso exemplo:

```
1 class ContaBanco {  
2 public:  
3     void exibirMensagem() {  
4         std::cout << "Conta bancaria criada.\n";  
5     }  
6 };
```

Há vários elementos importantes nesse pequeno exemplo:

- `class` é a palavra-chave utilizada para **declarar** uma classe;
- `ContaBanco` é o nome do **novo tipo** definido pelo programador;
- `public` especifica que os membros seguintes podem ser acessados pelo **código externo**, respeitando as regras da linguagem;
- `void` indica que a função não retorna um valor;
- `exibirMensagem` é o nome da **função-membro**;
- os parênteses `()` representam a **lista de parâmetros** da função;
- o bloco delimitado por `{` e `}` contém a **implementação** da função-membro;
- o ponto e vírgula após a chave de fechamento encerra a declaração da classe.

NOTA

O ponto e vírgula após a chave de fechamento da classe é obrigatório. Ele encerra a declaração do tipo e não deve ser confundido com o ponto e vírgula utilizado para encerrar uma instrução individual.

2.2.2 O Especificador de Acesso `public`

O exemplo anterior introduziu o **especificador de acesso** `public`. Ele determina quais membros da classe podem ser utilizados diretamente pelo código que está **fora** da própria classe.

Neste primeiro exemplo, a função-membro `exibirMensagem` foi declarada na seção pública da classe:

```
1 class ContaBanco {  
2     public:  
3         void exibirMensagem() {  
4             std::cout << "Conta bancaria criada.\n";  
5         }  
6 };
```

Isso significa que um `objeto` de `ContaBanco` poderá `solicitar a execução dessa função` a partir de `código externo`.

É importante observar que `public` não é uma função e também não é um tipo de dado. Trata-se de um **especificador de acesso**, utilizado para controlar a `visibilidade dos membros` de uma classe.

Os demais especificadores de acesso, especialmente `private` e `protected`, serão estudados posteriormente, quando introduzirmos formalmente o `encapsulamento`.

2.2.3 Instanciação de um Objeto

Declarar uma classe `não cria`, por si só, um objeto. A classe define o `tipo`; é necessário declarar uma `variável` desse `tipo` para `criar uma instância`.

Podemos criar um `objeto` da classe `ContaBanco` da seguinte forma:

Código 2.1: Criação de um `objeto` da classe `ContaBanco`

```
1 ContaBanco conta;
```

Nesse caso, `ContaBanco` é o tipo e `conta` é o nome do objeto.

A relação pode ser visualizada da seguinte maneira:

Tabela 2.3: Distinção entre `classe`, `tipo` e `objeto`.

Elemento	Exemplo	Função no programa
Classe	ContaBanco	Define a <code>estrutura</code> e os <code>comportamentos</code> disponíveis para seus <code>objetos</code> .
Tipo	ContaBanco	Identifica a <code>categoria</code> à qual uma <code>variável</code> pertence.
Objeto	conta	Instância concreta da classe criada durante a execução do programa.

Assim como ocorre com tipos fundamentais, como `int` e `double`, podemos declarar variá-

veis utilizando um **tipo definido pelo programador**. A diferença é que, nesse caso, o tipo é uma classe.

```
1 int idade{25};  
2 double altura{1.75};  
3  
4 ContaBanco conta;
```

Nas duas primeiras declarações, `idade` e `altura` são variáveis de tipos fundamentais. Na terceira, `conta` é um objeto cujo tipo foi definido pela classe `ContaBanco`.

2.2.4 Invocação de uma Função-Membro com o Operador Ponto

Depois que o **objeto** foi criado, podemos solicitar a **execução de uma função pública** pertencente à **classe** utilizando o **operador ponto** (`.`).

A sintaxe geral é:

```
1 objeto.funcaoMembro();
```

No nosso exemplo:

```
1 conta.exibirMensagem();
```

a expressão pode ser interpretada conceitualmente como:

$$\text{conta} \rightarrow . \rightarrow \text{exibirMensagem}()$$

O **objeto** `conta` é utilizado para acessar a **função-membro** `exibirMensagem` definida pela **classe** `ContaBanco`.

Essa forma de acesso é uma das características fundamentais da programação orientada a objetos em C++. O código externo não precisa conhecer a **implementação interna** da função para utilizá-la. Ele precisa apenas conhecer a **interface pública** disponibilizada pela **classe**.

2.2.5 Primeiro Programa Completo com uma Classe

Podemos reunir os elementos apresentados até aqui em um programa completo:

Código 2.2: code_cap2_conta_01.cpp

```
1 #include <iostream>
2
3 class ContaBanco {
4 public:
5     void exibirMensagem() {
6         std::cout << "Conta bancaria criada.\n";
7     }
8 };
9
10 int main()
11 {
12     ContaBanco conta;
13     conta.exibirMensagem();
14
15     return 0;
16 }
```

A execução desse programa produz:

```
Conta bancaria criada.
```

O fluxo de execução pode ser descrito em etapas:

1. O programa encontra a definição da classe `ContaBanco`;
2. a classe define uma função-membro pública denominada `exibirMensagem`;
3. a função `main` declara o objeto `conta`;
4. o objeto `conta` é utilizado para acessar `exibirMensagem` por meio do operador `.`;
5. a função-membro é executada e envia a mensagem para `std::cout`;
6. a função `main` termina retornando 0.

👍 Boa Prática de Programação 2.2: Nomes de Classes e Funções-Membro

Adote uma convenção consistente para a **nomenclatura**. Em materiais introdutórios e em muitos projetos C++, é comum utilizar a notação **PascalCase** para nomes de **classes**, como `ContaBanco`, e **camelCase** para **funções-membro**, como `exibirMensagem`. A convenção exata pode variar entre projetos; o mais importante é que seja definida e aplicada **consistentemente**.

⚠ Erro Comum de Programação 2.1

Confundir a declaração de uma classe com a criação de um objeto. A instrução `class ContaBanco { ... };` **define um tipo**, mas não cria automaticamente uma variável chamada `conta`. Para criar uma instância, é necessário **declarar explicitamente um objeto**, como em `ContaBanco conta;`.

⚠ Erro Comum de Programação 2.2

Tentar acessar uma **função-membro pública** sem utilizar um **objeto** ou sem obedecer à sintaxe de acesso apropriada. Para um objeto chamado `conta`, a chamada deve ser realizada como `conta.exibirMensagem();`. O operador ponto estabelece a relação entre o **objeto** e o **membro** que será acessado.

2.2.6 Uma Primeira Distinção entre Interface e Implementação

Embora o exemplo ainda seja pequeno, ele já permite introduzir uma distinção que será aprofundada posteriormente.

A declaração

```
1 void exibirMensagem();
```

descreve a existência de uma função-membro, enquanto sua implementação

```
1 void exibirMensagem() {
2     std::cout << "Conta bancaria criada.\n";
3 }
```

define **o que acontece** quando essa função é **executada**.

Em uma classe real, essa separação torna-se especialmente importante. A `interface` informa aos demais componentes do programa quais `operações` estão `disponíveis`, enquanto a `implementação` contém os `detalhes necessários para realizá-las`.

Nas próximas seções, a classe `ContaBanco` será progressivamente ampliada. Primeiro serão introduzidos `atributos` e o `ocultamento de dados`; em seguida, serão desenvolvidos métodos de acesso, `validação` e `construtores`. Posteriormente, a `interface` e a `implementação` serão separadas em `arquivos .h` e `.cpp`.

2.2.7 Exercício de Laboratório — Cadastro e Exibição de um Produto

Este exercício integra conceitos fundamentais apresentados no capítulo anterior (como declaração de variáveis, `tipos` de dados, `inicialização`, `atribuição`, `std::cout`, `std::cin`, operadores aritméticos e comentários) com os primeiros conceitos de `Programação Orientada a Objetos`.

Proposta do Exercício

Desenvolva um programa em C++ que modele um produto de uma loja por meio de uma classe denominada `Produto`. A classe deverá possuir `três funções-membro públicas`:

- `void definirDados();`
- `void aplicarDesconto(double percentual);`
- `void exibirDados();`

O programa deverá funcionar de acordo com o seguinte fluxo de execução:

1. Criar a classe `Produto`.
2. Declarar três atributos públicos: `codigo (int)`, `preco (double)` e `quantidade (int)`.
3. Instanciar um objeto denominado `produto` no escopo principal.
4. Solicitar ao usuário os dados e armazená-los via `definirDados()`.

5. Solicitar ao usuário um percentual de desconto.
6. Utilizar a função-membro `aplicarDesconto()` para calcular o novo preço, dado por:

$$\text{preço final} = \text{preço inicial} \times \left(1 - \frac{\text{percentual}}{100}\right) \quad (2.1)$$

7. Exibir os dados finais através do método `exibirDados()`.

👍 Boa Prática de Programação 2.3: Nomenclatura de Métodos e Atributos

Utilize nomes significativos e em notação *camelCase* para *funções-membro* e *variáveis de membros* (ex: `aplicarDesconto` e `percentualDesconto`). Isso amplia substancialmente a *clareza* do código.

Exemplo de Execução

Uma execução típica no terminal deverá apresentar o comportamento a seguir:

```
=== CADASTRO DE PRODUTO ===
Codigo: 101
Preco: 150.00
Quantidade: 3

Digite o percentual de desconto: 10

=== PRODUTO ===
Codigo: 101
Preco: R$ 135
Quantidade: 3
```

Solução Proposta

Abaixo encontra-se a implementação em C++ atendendo a todos os requisitos do laboratório:

Código 2.3: `lab_produto.cpp`

```
1 #include <iostream>
2
3 class Produto {
4 public:
5     int codigo;
6     double preco;
7     int quantidade;
8
9     void definirDados() {
10         std::cout << "Codigo: ";
11         std::cin >> codigo;
12
13         std::cout << "Preco: ";
14         std::cin >> preco;
15
16         std::cout << "Quantidade: ";
17         std::cin >> quantidade;
18     }
19
20     void aplicarDesconto(double percentual) {
21         preco = preco * (1.0 - percentual / 100.0);
22     }
23
24     void exibirDados() {
25         std::cout << "\n=== PRODUTO ===\n";
26         std::cout << "Codigo: " << codigo << '\n';
27         std::cout << "Preco: R$" << preco << '\n';
28         std::cout << "Quantidade: " << quantidade << '\n';
29     }
30 };
31
32 int main()
33 {
34     Produto produto;
35     double percentualDesconto{0.0};
36
37     std::cout << "=== CADASTRO DE PRODUTO ===\n";
38
```

```

39 // Leitura dos dados do produto
40 produto.definirDados();
41
42 // Leitura do percentual de desconto
43 std::cout << "\nDigite o percentual de desconto: ";
44 std::cin >> percentualDesconto;
45
46 // Aplicacao do desconto ao preco
47 produto.aplicarDesconto(percentualDesconto);
48
49 // Exibicao dos dados atualizados
50 produto.exibirDados();
51
52 return 0;
53 }

```

Análise Didática da Solução

A análise detalhada do 2.3 permite fixar a distinção entre os novos conceitos de Orientação a Objetos:

- **Classe** (class Produto): É o tipo de dado abstrato projetado pelo programador.
- **Objeto** (Produto produto): É a instância concreta criada na memória durante a execução.
- **Atributos** (codigo, preco, quantidade): São os membros de dados que representam o estado interno do objeto.
- **Funções-Membro** (definirDados, etc.): Definem os comportamentos e operações associadas ao objeto.

A chamada `produto.aplicarDesconto(percentualDesconto)` une esses fundamentos: o objeto (`produto`) utiliza o operador de acesso (`.`) para invocar a função-membro correspondente.

⚠ Erro Comum de Programação 2.3: Atributos **sem** Encapsulamento

Manter `atributos` na seção `public` permite que `qualquer parte do código` altere o `estado do objeto` livremente (por exemplo, atribuindo `produto.preco = -500.0`). Nas próximas seções, resolveremos esse problema utilizando o `modificador private` e funções de acesso (*getters/setters*).

Desafio Adicional — Cálculo do Valor em Estoque

Como extensão prática do exercício, adicione à classe a função-membro `calcularValorEstoque()` que retorna o valor monetário **total** do lote:

```
1 double calcularValorEstoque() {  
2     return preco * quantidade;  
3 }
```

No escopo da função `main`, a chamada e exibição do resultado podem ser feitas como demonstrado a seguir:

```
1 double valorEstoque = produto.calcularValorEstoque();  
2 std::cout << "Valor total do estoque: R$ " << valorEstoque << '\n';
```

Essa adição demonstra de forma simples que funções-membro não se limitam ao tipo de retorno `void`, podendo processar atributos internos e devolver valores calculados ao chamador.

2.3 Membros de Dados, Funções de Acesso e Encapsulamento

No exercício anterior, a classe `Produto` possuía seus membros de dados declarados na seção `public`. Embora essa organização seja sintaticamente válida, ela apresenta um problema importante de projeto: qualquer código externo que tenha acesso ao objeto pode modificar **diretamente** seu estado interno.

Considere, por exemplo, o seguinte código:

```
1 Produto produto;  
2  
3 produto.preco = -150.0;  
4 produto.quantidade = -20;
```

O compilador não possui, nesse caso, nenhuma razão para rejeitar essas atribuições. Os valores pertencem aos tipos `double` e `int`, respectivamente, e as operações de atribuição

são sintaticamente válidas.

Entretanto, do ponto de vista do **domínio do problema**, uma situação em que um produto possui preço negativo ou quantidade negativa provavelmente não é válida.

Esse exemplo evidencia uma questão fundamental da **programação orientada a objetos**:

*Uma **classe** não deve apenas armazenar **dados**; ela também deve **proteger** as **regras** que determinam quais **estados** são válidos para seus **objetos**.*

Para alcançar esse objetivo, C++ oferece mecanismos de **controle de acesso** aos **membros** de uma classe. Entre eles, o **especificador private** possui papel **central** na implementação do **encapsulamento**.

2.3.1 Ocultamento de Dados com private

O **especificador de acesso private** determina que os **membros** declarados nessa seção somente podem ser acessados **diretamente** por **funções** e outras **entidades** que possuem os **privilegios apropriados** definidos pelas **regras da linguagem**.

Em uma classe, podemos separar explicitamente os membros públicos dos membros privados:

```
1 class Produto {  
2     private:  
3         int codigo;  
4         double preco;  
5         int quantidade;  
6  
7     public:  
8         void exibirDados() {  
9             std::cout << "Codigo: " << codigo << '\n';  
10            std::cout << "Preco: " << preco << '\n';  
11            std::cout << "Quantidade: " << quantidade << '\n';  
12        }  
13    };
```

Nesse exemplo, os três **membros de dados** pertencem à seção `private`. Consequentemente, uma instrução externa como:

```
1 Produto produto;  
2  
3 produto.preco = 150.0;
```

não é permitida.

O acesso direto ao membro `preco` foi **restringido** pela própria classe. O código externo deve utilizar as operações **públicas** que a classe **disponibilizar** para consultar ou modificar seu estado.

NOTA

Em uma declaração iniciada pela palavra-chave `class`, os **membros** posicionados **antes do primeiro especificador de acesso** possuem, por padrão, **acesso `private`**. Mesmo assim, é uma boa prática declarar explicitamente a seção `private` quando o objetivo didático ou de projeto é deixar evidente quais membros representam o estado interno da classe.

2.3.2 Encapsulamento: Estado Interno e Interface Pública

O encapsulamento pode ser compreendido como a **organização de dados e operações relacionados** em uma **unidade**, acompanhada do **controle sobre como esse estado** pode ser **acessado** ou **modificado**.

A classe passa, portanto, a estabelecer uma fronteira entre:

- **estado interno:** **dados** que representam a situação **atual** do **objeto** e que não devem ser manipulados diretamente pelo **código externo**;
- **interface pública:** **operações** disponibilizadas pela classe para que outras partes do programa possam interagir com o objeto.

Essa separação pode ser representada conceitualmente da seguinte forma:

Tabela 2.4: Separação entre estado interno e interface pública.

Elemento	Exemplo	Função
Estado interno	<code>double preco;</code>	Armazena uma característica do objeto e permanece protegido do acesso direto externo.
Operação pública	<code>void setPreco(double valor);</code>	Permite solicitar uma alteração do estado por meio de uma interface controlada pela classe .
Consulta pública	<code>double getPreco() const;</code>	Permite consultar o estado sem conceder acesso direto ao membro interno.

O objetivo não é impedir que o estado de um objeto seja alterado. O objetivo é fazer com que a alteração ocorra de maneira **controlada**.

Essa distinção é essencial. Uma classe bem **encapsulada** não precisa tornar seus dados **imutáveis**; ela deve estabelecer **regras claras** para as operações que podem modificar esses dados.

2.3.3 Funções de Acesso: **Getters**

Quando um **membro de dados** é **privado**, uma forma convencional de **permitir sua consulta** é fornecer uma **função-membro pública** denominada, por convenção, **getter**. Considere:

```

1 class Produto {
2     private:
3         double preco;
4     public:
5         double getPreco() const {
6             return preco;
7         }
8 };

```

A função `getPreco()` retorna o valor atualmente armazenado em `preco`. A palavra-chave `const`, posicionada após os parênteses da função, indica que essa função-membro **não modifica o estado** observável do objeto.

Assim, podemos consultar o preço por meio da **interface pública**:

```
1 Produto produto;  
2  
3 double valor = produto.getPreco();
```

Observe a diferença em relação ao **acesso direto**:

```
1 // Acesso direto: nao permitido se preco for private.  
2 produto.preco;  
3  
4 // Acesso por meio da interface publica.  
5 produto.getPreco();
```

No segundo caso, o código externo não conhece nem manipula diretamente o membro privado. Ele solicita à própria classe o valor que deseja consultar.

2.3.4 Funções de Modificação: **Setters**

Uma **função pública** utilizada para **modificar um membro privado** é frequentemente denominada **setter**.

A implementação mais simples seria:

```
1 class Produto {  
2     private:  
3         double preco;  
4  
5     public:  
6         void setPreco(double novoPreco) {  
7             preco = novoPreco;  
8         }  
9 };
```

Agora, o código externo não modifica diretamente `preco`. Em vez disso, solicita à classe que realize a alteração:

```
1 Produto produto;
2
3 produto.setPreco(150.0);
```

A diferença pode parecer pequena, mas possui uma **consequência** importante: a própria **classe** passa a controlar o **ponto** em que seu estado é **modificado**.

Isso permite acrescentar **regras de validação** sem alterar a forma geral de utilização do **objeto**.

2.3.5 Validação do Estado no **Setter**

Considere que, para o domínio da classe `Produto`, o preço não possa ser negativo. Um **setter** pode incorporar essa regra:

```
1 void setPreco(double novoPreco) {
2     if (novoPreco >= 0.0) {
3         preco = novoPreco;
4     }
5 }
```

Agora, uma tentativa de atribuir um valor negativo **não altera o estado do objeto**:

```
1 produto.setPreco(150.0);    // Valor aceito.
2 produto.setPreco(-20.0);    // Valor rejeitado.
```

A classe passou a assumir responsabilidade pela **integridade** de seu próprio estado.

Entretanto, a estratégia utilizada acima possui uma **limitação**: quando o valor é **rejeitado**, o programa não informa ao chamador que a operação falhou. Dependendo dos requisitos da aplicação, uma solução mais apropriada poderia ser **retornar um valor lógico**, lançar uma exceção ou utilizar outro mecanismo de **tratamento de erro**.

Neste momento, o objetivo é apenas compreender o princípio fundamental: **a validação deve ocorrer no ponto em que o estado é modificado**.

2.3.6 Aplicação à Classe Produto

Podemos agora reconstruir a classe utilizada no exercício de laboratório, aplicando o princípio de **ocultamento de dados**.

```
1 class Produto {
2 private:
3     int codigo;
4     double preco;
5     int quantidade;
6 public:
7     void setCodigo(int novoCodigo) {
8         codigo = novoCodigo;
9     }
10
11     int getCodigo() const {
12         return codigo;
13     }
14
15     void setPreco(double novoPreco) {
16         if (novoPreco >= 0.0) {
17             preco = novoPreco;
18         }
19     }
20
21     double getPreco() const {
22         return preco;
23     }
24
25     void setQuantidade(int novaQuantidade) {
26         if (novaQuantidade >= 0) {
27             quantidade = novaQuantidade;
28         }
29     }
30
31     int getQuantidade() const {
32         return quantidade;
33     }
34 };
```

Agora os **membros de dados** não podem mais ser modificados **diretamente** pelo código externo.

Por exemplo:

```
1 Produto produto;
2
3 produto.setCodigo(101);
4 produto.setPreco(150.0);
5 produto.setQuantidade(3);
6
7 std::cout << "Codigo: "
8           << produto.getCodigo() << '\n';
9
10 std::cout << "Preco: R$ "
11          << produto.getPreco() << '\n';
12
13 std::cout << "Quantidade: "
14          << produto.getQuantidade() << '\n';
```

O código que utiliza o objeto conhece sua interface pública, mas não precisa conhecer a representação interna de seus dados.

2.3.7 Encapsulamento não Significa Apenas Criar Getters e Setters

É comum encontrar uma interpretação excessivamente simplificada segundo a qual encapsulamento significaria apenas declarar todos os atributos como `private` e criar automaticamente um *getter* e um *setter* para cada um deles.

Essa interpretação é inadequada.

O objetivo do encapsulamento é controlar e organizar o acesso ao estado e ao comportamento de uma abstração. Nem todo dado precisa necessariamente possuir um *setter*, e algumas classes podem não disponibilizar diretamente um *getter*.

Por exemplo, se a quantidade de produtos só puder ser alterada por meio de uma operação de compra ou venda, pode ser conceitualmente mais adequado possuir funções como:

```
1 void adicionarEstoque(int quantidade);
```

```
2 void registrarVenda(int quantidade);
```

em vez de simplesmente disponibilizar:

```
1 void setQuantidade(int quantidade);
```

A primeira abordagem permite que a própria classe preserve regras de negócio, como impedir uma venda superior ao estoque disponível.

Essa distinção será importante à medida que o projeto evoluir: uma boa classe deve expor operações que representem responsabilidades do domínio, e não simplesmente reproduzir mecanicamente seus atributos como funções de acesso.

Boa Prática de Programação 2.4

Declare como `private` os membros de dados que representam o estado interno da classe e forneça uma interface pública coerente com as operações que os usuários da classe realmente precisam realizar. Não crie automaticamente um `setter` para cada atributo apenas por convenção.

Erro Comum de Programação 2.4: Atributos Públicos

Declarar indiscriminadamente os membros de dados como `public` permite que qualquer código com acesso ao objeto altere diretamente seu estado, inclusive para valores que violem as regras do domínio.

Por exemplo:

```
1 produto.preco = -500.0;  
2 produto.quantidade = -10;
```

A linguagem não considera essas atribuições inválidas apenas porque os valores são inadequados para o problema. Cabe ao projeto da classe impedir que estados inválidos sejam produzidos por sua interface.

⚠ Erro Comum de Programação 2.5: Confundir Encapsulamento com Ocultamento Absoluto

Encapsular não significa esconder completamente um objeto do restante do programa. Uma classe encapsulada continua oferecendo uma interface pública. O princípio consiste em ocultar detalhes de implementação e controlar o acesso ao estado interno por meio de operações apropriadas.

NOTA

As convenções *getter* e *setter* são nomes adotados pela comunidade de programação para funções de acesso e modificação. Eles não são palavras-chave da linguagem C++ e não possuem significado especial para o compilador.

2.4 Construtores de Classe

Na seção anterior, vimos que uma classe pode proteger seu estado interno por meio do especificador `private` e disponibilizar uma interface pública para que o código externo interaja com seus objetos.

Entretanto, surge uma questão importante: **em que momento os membros de dados de um objeto devem receber seus valores iniciais?**

Considere novamente uma classe que representa um produto:

Código 2.4: Criação de um objeto antes da inicialização

```
1 class Produto
2 {
3     private:
4         std::string nome;
5         double preco{0.0};
6
7     public:
8         void setNome(std::string novoNome)
9         {
```

```
10     nome = novoNome;
11 }
12
13 void setPreco(double novoPreco)
14 {
15     if (novoPreco >= 0.0)
16     {
17         preco = novoPreco;
18     }
19 }
20 };
```

Um objeto dessa classe pode ser criado e posteriormente configurado:

```
1 Produto produto;
2
3 produto.setNome("Notebook");
4 produto.setPreco(3500.00);
```

Embora esse código seja válido, existe um intervalo conceitual entre a criação do objeto e sua configuração completa. Em classes mais complexas, essa situação pode ser problemática, principalmente quando determinados dados são obrigatórios para que o objeto seja considerado válido.

A linguagem C++ oferece um mecanismo específico para resolver esse problema: o **construtor**.

2.4.1 O Papel do Construtor

Um **construtor** é uma função-membro especial responsável por estabelecer o estado inicial de um objeto no momento em que ele é criado.

Diferentemente de uma função-membro comum, o construtor possui algumas características particulares:

- possui o mesmo nome da classe;
- não possui tipo de retorno, nem mesmo `void`;

- é executado automaticamente durante a construção do objeto;
- pode receber parâmetros;
- pode inicializar os membros de dados do objeto.

Uma classe simples pode possuir um construtor definido da seguinte forma:

Código 2.5: Primeiro construtor

```
1 class Produto
2 {
3     private:
4         std::string nome;
5         double preco;
6
7     public:
8         Produto()
9         {
10             nome = "Produto sem nome";
11             preco = 0.0;
12         }
13 };
```

Agora, quando um objeto é criado:

```
1 Produto produto;
```

o construtor `Produto()` é chamado automaticamente.

Não é necessário realizar uma chamada explícita como:

```
1 // Nao fazemos isso:
2 produto.Produto();
```

O construtor faz parte do processo de construção do objeto e é acionado automaticamente pela linguagem.

NOTA

Um construtor não é chamado como uma função-membro comum. Sua execução está associada ao processo de criação do objeto.

2.4.2 Construtor Padrão (*Default Constructor*)

Um construtor que pode ser chamado sem argumentos é denominado **construtor padrão** (*default constructor*).

Por exemplo:

```
1 class Produto
2 {
3 public:
4     Produto()
5     {
6         // Inicializacao do objeto
7     }
8 };
```

Nesse caso, a construção pode ser realizada simplesmente por:

```
1 Produto produto;
```

É importante distinguir dois conceitos que frequentemente são confundidos.

O primeiro é o **construtor padrão fornecido implicitamente pelo compilador**. O segundo é um **construtor padrão definido pelo programador**.

Quando uma classe não declara nenhum construtor, o compilador pode fornecer implicitamente um construtor padrão, desde que as regras da linguagem permitam sua geração.

Por outro lado, quando o programador declara explicitamente:

```
1 Produto()
2 {
3     // ...
4 }
```

o construtor passa a fazer parte da definição explícita da classe.

Essa distinção torna-se especialmente importante quando a classe possui regras específicas para a inicialização de seus membros.

2.4.3 Construtores Parametrizados

Um construtor também pode receber argumentos. Nesse caso, os valores fornecidos durante a criação do objeto são utilizados para estabelecer seu estado inicial.

Considere:

Código 2.6: Construtor parametrizado

```
1 class Produto
2 {
3     private:
4         std::string nome;
5         double preco;
6
7     public:
8         Produto(std::string nomeInicial, double precoInicial)
9         {
10             nome = nomeInicial;
11             preco = precoInicial;
12         }
13 };
```

Agora, o objeto pode ser criado já contendo os dados iniciais:

```
1 Produto produto("Notebook", 3500.00);
```

Nesse caso, a sequência conceitual é diferente daquela observada anteriormente:

1. o programa solicita a criação de um objeto `Produto`;
2. o construtor recebe os argumentos fornecidos;
3. os membros do objeto são inicializados;

4. o objeto passa a existir com seu estado inicial estabelecido.

Isso permite que a criação do objeto e a definição de seu estado inicial sejam tratadas como uma única operação.

Código 2.7: Construtor parametrizado em uso

```
1 Produto produto1("Notebook", 3500.00);  
2 Produto produto2("Teclado", 250.00);
```

Os dois objetos pertencem à mesma classe, mas possuem estados iniciais independentes.

2.4.4 Lista de Inicialização de Membros

Embora seja possível atribuir valores aos membros dentro do corpo do construtor, essa não é a forma preferencial de inicialização em C++ moderno.

Considere novamente:

```
1 class Produto  
2 {  
3     private:  
4         std::string nome;  
5         double preco;  
6  
7     public:  
8         Produto(std::string nomeInicial, double precoInicial)  
9         {  
10             nome = nomeInicial;  
11             preco = precoInicial;  
12         }  
13 };
```

Nesse código, os membros são inicialmente construídos e, posteriormente, recebem atribuições dentro do corpo do construtor.

C++ fornece uma forma mais apropriada para realizar essa inicialização: a **lista de inicialização de membros** (*member initializer list*).

A mesma classe pode ser escrita como:

Código 2.8: Construtor utilizando lista de inicialização

```
1 class Produto
2 {
3     private:
4         std::string nome;
5         double preco;
6
7     public:
8         Produto(std::string nomeInicial, double precoInicial)
9             : nome{nomeInicial},
10              preco{precoInicial}
11         {
12         }
13 };
```

A lista iniciada por dois-pontos (:) aparece entre a assinatura do construtor e o seu corpo.

Cada elemento da lista possui a forma:

```
1 membro{valor}
```

No exemplo:

```
1 : nome{nomeInicial},
2   preco{precoInicial}
```

o membro `nome` é inicializado com `nomeInicial` e o membro `preco` é inicializado com `precoInicial`.

Essa distinção entre **inicialização** e **atribuição** é fundamental em C++.

NOTA

A lista de inicialização não representa simplesmente uma forma abreviada de atribuir valores dentro do construtor. Os membros são efetivamente **inicializados** antes da execução do corpo do construtor.

2.4.5 Ordem de Inicialização dos Membros

Um detalhe importante da linguagem C++ é que os membros de dados são inicializados na **ordem em que foram declarados na classe**, e não na ordem em que aparecem na lista de inicialização do construtor [?].

Considere:

```
1 class Exemplo
2 {
3     private:
4         int primeiro;
5         int segundo;
6
7     public:
8         Exemplo()
9             : segundo{20},
10              primeiro{10}
11         {
12         }
13 };
```

Embora a lista apresente primeiro segundo e depois primeiro, a linguagem inicializa os membros na ordem de declaração:

1. primeiro;
2. segundo.

Por essa razão, recomenda-se que a lista de inicialização siga a mesma ordem em que os membros aparecem na definição da classe.

Boa Prática de Programação 2.5: Inicialização de Membros

Sempre que possível, inicialize os membros de dados por meio da lista de inicialização do construtor ou diretamente na declaração da classe. Além de expressar claramente a intenção do código, essa abordagem evita inicialização seguida de atribuição e é obrigatória para determinados tipos de membros, como referências e

objetos que não possuem construtor padrão.

2.4.6 Inicialização Direta na Declaração da Classe

C++ moderno também permite estabelecer um valor inicial diretamente na declaração de um membro de dados.

Por exemplo:

Código 2.9: Inicializador padrão de membro

```
1 class Produto
2 {
3 private:
4     std::string nome{"Produto sem nome"};
5     double preco{0.0};
6 };
```

Esses valores funcionam como inicializadores padrão dos membros e são utilizados quando o construtor não fornece uma inicialização específica para esses membros.

Essa técnica é particularmente útil quando existe um valor padrão que faz sentido para todos os objetos da classe.

Por exemplo, um contador pode naturalmente começar em zero:

```
1 class Contador
2 {
3 private:
4     int valor{0};
5
6 public:
7     void incrementar()
8     {
9         ++valor;
10    }
11 };
```

Nesse caso, a classe expressa diretamente uma propriedade importante de seu estado: um novo contador começa com valor zero.

2.4.7 Construtor Padrão e Construtor Parametrizado na Mesma Classe

Uma classe pode fornecer mais de um construtor, desde que suas assinaturas sejam suficientemente diferentes para permitir a resolução da sobrecarga.

Por exemplo:

Código 2.10: Sobrecarga de construtores

```
1 class Produto
2 {
3 private:
4     std::string nome{"Produto sem nome"};
5     double preco{0.0};
6
7 public:
8     Produto()
9     {
10    }
11
12     Produto(std::string nomeInicial, double precoInicial)
13         : nome{nomeInicial},
14           preco{precoInicial}
15     {
16    }
17 };
```

Agora existem duas possibilidades:

```
1 Produto produto1;
2
3 Produto produto2("Notebook", 3500.00);
```

No primeiro caso, é utilizado o construtor sem argumentos. No segundo, é utilizado o construtor que recebe nome e preço.

Essa capacidade de fornecer diferentes formas de construção é denominada **sobrecarga de construtores** e será explorada progressivamente ao longo do capítulo.

2.4.8 Construtores e Validação do Estado Inicial

O construtor também pode desempenhar um papel importante na manutenção da validade de um objeto desde o instante em que ele é criado.

Considere uma classe que representa um produto cujo preço não pode ser negativo:

Código 2.11: Construtor com validação

```
1 class Produto
2 {
3     private:
4         std::string nome;
5         double preco;
6
7     public:
8         Produto(std::string nomeInicial, double precoInicial)
9             : nome{nomeInicial},
10              preco{0.0}
11         {
12             if (precoInicial >= 0.0)
13             {
14                 preco = precoInicial;
15             }
16         }
17 };
```

O construtor estabelece um valor inicial válido para `preco` e só permite a utilização do valor recebido quando ele satisfaz a regra definida pela classe.

Esse exemplo introduz uma ideia importante para o desenvolvimento de classes robustas: o objeto deve procurar manter seus dados em um estado coerente durante todo o seu ciclo de vida.

Em projetos maiores, essas regras de consistência são frequentemente denominadas **in-**

variantes da classe.

2.4.9 Exercício Prático Orientado: Classe Livro com Construtores

Para consolidar os conceitos apresentados, estenda o exercício da classe Livro desenvolvido anteriormente.

A versão anterior permitia criar um livro e posteriormente preencher seus dados por meio de funções-membro. Agora, o objetivo é utilizar **construtores** para que o objeto possa ser criado já com seu estado inicial definido.

Especificação

Implemente a classe Livro contendo:

- `titulo`, do tipo `std::string`;
- `autor`, do tipo `std::string`;
- `anoPublicacao`, do tipo `int`.

Todos os membros de dados deverão ser privados.

A classe deverá possuir:

1. um construtor padrão;
2. um construtor parametrizado que receba título, autor e ano de publicação;
3. funções-membro para consultar os dados do livro.

O construtor parametrizado deverá utilizar uma **lista de inicialização de membros**.

O ano de publicação deverá ser considerado válido somente quando for maior que zero.

O programa principal deverá criar:

- um objeto utilizando o construtor padrão;
- um objeto utilizando o construtor parametrizado.

Depois, deverá exibir os dados dos dois objetos.

Boa Prática de Programação 2.6: Desafio adicional

Crie um terceiro construtor que receba somente o título e o autor, utilizando um valor padrão apropriado para o ano de publicação. Compare essa solução com a utilização de argumentos padrão e discuta qual abordagem torna a interface da classe mais clara.

Objetivos de aprendizagem

Ao concluir o exercício, o estudante deverá ser capaz de:

- explicar a finalidade de um construtor;
- diferenciar construtor padrão de construtor parametrizado;
- criar objetos utilizando diferentes construtores;
- utilizar listas de inicialização de membros;
- distinguir inicialização de atribuição;
- compreender a ordem de inicialização dos membros;
- utilizar construtores para estabelecer o estado inicial de um objeto.

Erro Comum de Programação 2.6: Confundir inicialização com atribuição

Evite implementar o construtor simplesmente atribuindo valores aos membros dentro de seu corpo quando uma lista de inicialização puder ser utilizada. A inicialização por meio da lista é realizada antes da execução do corpo do construtor e constitui a forma idiomática de inicializar membros em C++.

⚠ Erro Comum de Programação 2.7: Ignorar a ordem de declaração dos membros

Não suponha que os membros serão inicializados na ordem apresentada na lista de inicialização. A ordem efetiva é determinada pela ordem de declaração dos membros na classe. Para evitar confusão e advertências do compilador, escreva a lista de inicialização na mesma ordem da declaração dos membros.

2.5 Separação de Interface e Implementação

À medida que um programa C++ cresce, continuar com todas as declarações e implementações em um único arquivo torna o código mais difícil de organizar, reutilizar e manter. Uma prática fundamental na construção de programas orientados a objetos consiste em separar a **interface** de uma classe de sua **implementação**.

A interface descreve **o que** a classe oferece ao restante do programa, enquanto a implementação especifica **como** essas operações são realizadas. Essa separação permite que o código que utiliza uma classe conheça seus serviços públicos sem precisar conhecer os detalhes internos responsáveis por sua execução.

Em projetos C++, essa organização é normalmente realizada por meio de dois tipos de arquivo:

- arquivos de cabeçalho (.h ou .hpp), utilizados principalmente para declarar a interface da classe;
- arquivos de implementação (.cpp), utilizados para definir as funções-membro e outras operações que constituem a implementação da classe.

Essa organização constitui uma das formas mais tradicionais de estruturar programas C++ e será utilizada progressivamente nos exemplos deste capítulo.

2.5.1 O Arquivo de Cabeçalho como Interface da Classe

Considere novamente a classe `ContaBanco`, introduzida nas seções anteriores. Em vez de concentrar toda a classe em um único arquivo, podemos colocar sua declaração em um arquivo denominado `ContaBanco.h`.

Nesse arquivo, apresentamos ao restante do programa quais operações estão disponíveis e quais dados fazem parte da estrutura interna da classe. As funções-membro podem ser declaradas sem que seus corpos sejam implementados nesse momento.

Código 2.12: `ContaBanco.h`

```
1 #ifndef CONTABANCO_H
2 #define CONTABANCO_H
3 #include <string>
4
5 class ContaBanco
6 {
7     private:
8         std::string nomeTitular;
9
10    public:
11        ContaBanco(const std::string& nome);
12
13        void setNomeTitular(const std::string& nome);
14        std::string getNomeTitular() const;
15 };
16 #endif
```

Observe que o arquivo apresenta a **declaração** da classe e de suas funções-membro, mas não contém o corpo dessas funções. Por exemplo,

```
1 void setNomeTitular(const std::string& nome);
```

informa que a classe oferece uma função-membro chamada `setNomeTitular`, que recebe uma referência constante para `std::string` e não retorna um valor (`void`).

O ponto e vírgula encerra essa declaração. O corpo da função será fornecido posteriormente no arquivo de implementação.

NOTA

O arquivo de cabeçalho funciona, portanto, como uma espécie de **contrato da classe**: ele informa quais tipos, funções e operações estão disponíveis para o código cliente, sem necessariamente revelar como essas operações são implementadas.

2.5.2 O Arquivo de Implementação

As definições das funções-membro podem ser transferidas para um arquivo `.cpp`. Para a classe apresentada anteriormente, utilizaremos o arquivo `ContaBanco.cpp`.

Código 2.13: `ContaBanco.cpp`

```

1  #include "ContaBanco.h"
2
3  ContaBanco::ContaBanco(const std::string& nome)
4      : nomeTitular{nome}
5  {
6  }
7
8  void ContaBanco::setNomeTitular(const std::string& nome)
9  {
10     nomeTitular = nome;
11 }
12
13 std::string ContaBanco::getNomeTitular() const
14 {
15     return nomeTitular;
16 }

```

A primeira linha,

```
1  #include "ContaBanco.h"
```

faz com que a declaração da classe esteja disponível durante a compilação desse arquivo de implementação.

As funções-membro são então definidas fora do corpo da classe utilizando o **operador de resolução de escopo** (`::`). Por exemplo:

1

void ContaBanco::setNomeTitular(const std::string& nome)

deve ser interpretado como:

“A função setNomeTitular, pertencente à classe ContaBanco, possui esta implementação.”

O identificador ContaBanco especifica a classe e o operador :: estabelece a relação entre a classe e a função-membro.

NOTA

A declaração no arquivo de cabeçalho e a definição no arquivo .cpp devem ser compatíveis. Se a assinatura da função for modificada em um arquivo e não for atualizada no outro, poderão ocorrer erros durante a compilação.

2.5.3 O Programa Cliente

Uma vez separadas a interface e a implementação, podemos criar um terceiro arquivo responsável por utilizar a classe. Esse arquivo será chamado, neste exemplo, de main.cpp.

Código 2.14: main.cpp

1

#include <iostream>

2

#include "ContaBanco.h"

3

int main()

4

{

5

ContaBanco conta{"Maria"};

6

std::cout << "Titular: "

7

<< conta.getNomeTitular()

8

<< '\n';

9

conta.setNomeTitular("Joao");

10

std::cout << "Novo titular: "

11

<< conta.getNomeTitular()

12

<< '\n';

13

return 0;

14

}

Temos agora três responsabilidades distintas:

- `ContaBanco.h`: declara a interface da classe;
- `ContaBanco.cpp`: implementa as funções-membro;
- `main.cpp`: utiliza a classe para executar o programa.

Essa separação é importante porque o programa cliente não precisa conhecer os detalhes da implementação de `ContaBanco`. Ele precisa apenas conhecer sua interface pública.

2.5.4 Guardas de Inclusão

Arquivos de cabeçalho podem ser incluídos, direta ou indiretamente, por diversos arquivos do projeto. Isso pode fazer com que o mesmo cabeçalho seja processado mais de uma vez durante uma unidade de tradução.

Para evitar a redefinição da classe, uma técnica tradicional consiste em utilizar **guardas de inclusão** (*include guards*).

No exemplo apresentado anteriormente:

```
1 #ifndef CONTABANCO_H
2 #define CONTABANCO_H
3 // Declaração da classe
4 #endif
```

o funcionamento pode ser entendido em três etapas:

1. `#ifndef CONTABANCO_H` verifica se o identificador `CONTABANCO_H` ainda não foi definido;
2. `#define CONTABANCO_H` define o identificador caso ele ainda não exista;
3. `#endif` encerra a região protegida pela condição.

Na primeira inclusão, o conteúdo da classe é processado. Em uma inclusão posterior do mesmo cabeçalho durante a mesma unidade de tradução, a macro já estará definida e o conteúdo protegido não será processado novamente.

❗ Erro Comum de Programação 2.8: Redefinição de Classe

Esquecer as guardas de inclusão em um arquivo de cabeçalho pode fazer com que a declaração da mesma classe seja processada mais de uma vez durante uma unidade de tradução. Isso pode resultar em erros de redefinição de classe.

As guardas devem envolver todo o conteúdo relevante do cabeçalho.

2.5.5 A Diretiva #pragma once

Uma alternativa amplamente utilizada para evitar inclusões múltiplas é a diretiva:

```
1 #pragma once
```

Assim, o cabeçalho poderia ser escrito de forma mais concisa:

Código 2.15: ContaBanco.h utilizando pragma once

```
1 #pragma once
2
3 #include <string>
4
5 class ContaBanco
6 {
7     private:
8         std::string nomeTitular;
9
10    public:
11        ContaBanco(const std::string& nome);
12
13        void setNomeTitular(const std::string& nome);
14        std::string getNomeTitular() const;
15 };
```

A diretiva informa ao compilador que o arquivo deve ser incluído apenas uma vez em uma unidade de tradução.

👍 Boa Prática de Programação 2.7: Organização dos Arquivos

Em projetos C++ modernos, mantenha a **interface pública** da **classe** em um **arquivo de cabeçalho** e suas **implementações**, sempre que apropriado, em um arquivo de origem separado. Essa organização facilita a **leitura**, a **manutenção** e a **reutilização** das **classes**.

Para projetos novos, `#pragma once` é uma alternativa **prática** às guardas tradicionais de inclusão, desde que o ambiente de compilação adotado ofereça suporte a essa diretiva.

2.5.6 Compilação de um Projeto com Múltiplos Arquivos

A separação entre **interface** e **implementação** também altera a forma como o programa é **compilado**. Agora temos mais de um arquivo-fonte participando da construção do executável.

Considerando a seguinte estrutura:

```
projeto/  
|-- main.cpp  
|-- ContaBanco.cpp  
|-- ContaBanco.h
```

podemos utilizar o **compilador g++** para compilar os arquivos de implementação e gerar o executável:

```
g++ main.cpp ContaBanco.cpp -o programa
```

No Linux e no macOS, o programa pode então ser executado com:

```
./programa
```

No Windows, utilizando o GCC/MinGW, o executável poderá ser executado com:

```
programa.exe
```

É importante perceber que `ContaBanco.h` não aparece como argumento de compilação nesse comando. O `cabeçalho` é incluído pelo `pré-processador` por meio das diretivas `#include` presentes nos arquivos-fonte.

NOTA

Em uma construção real de software, compilação e vinculação (*linking*) são etapas conceitualmente distintas. Cada arquivo-fonte pode ser compilado em uma unidade de tradução e, posteriormente, os arquivos-objeto resultantes são vinculados para formar o executável. O comando `g++` pode automatizar essas etapas quando recebe múltiplos arquivos-fonte em uma única invocação.

2.5.7 Exercício Prático: Separando uma Classe em Arquivos

Lista de Exercícios

A classe `Produto`, desenvolvida nas seções anteriores, encontra-se atualmente implementada em um único arquivo.

Sua tarefa é **reorganizar o programa** utilizando a separação entre `interface` e `implementação`.

Crie os seguintes arquivos:

- (a) `Produto.h`, contendo a `declaração` da classe;
- (b) `Produto.cpp`, contendo a `implementação` das `funções-membro`;
- (c) `main.cpp`, contendo a função `main` e utilizando a `classe Produto`.

A classe deverá possuir, no mínimo:

- um **atributo privado** para armazenar o **nome** do produto;

- um atributo privado para armazenar o **preço**;
- um **construtor**;
- uma função-membro para **obter** o nome;
- uma função-membro para **obter** o preço;
- uma função-membro para **alterar** o preço, rejeitando valores **negativos**.

Utilize uma **guarda de inclusão** ou `#pragma once` no **arquivo de cabeçalho**.

Por fim, compile os três arquivos utilizando `g++` e execute o programa pelo terminal.

Boa Prática de Programação 2.8

Não coloque **implementações** de **funções-membro** no arquivo `main.cpp` apenas para **facilitar a compilação** durante os primeiros testes. O objetivo da separação é estabelecer **responsabilidades** claras entre os **arquivos** e preparar o **projeto** para uma organização modular.

2.6 Tipos de Dados do C++ e a Classe `std::string`

Até este ponto, os exemplos utilizaram tipos fundamentais da linguagem, como `int` e `double`, além de **classes** da **Biblioteca Padrão**, como `std::string`. Compreender essa **distinção** é importante para entender como o **sistema de tipos** do C++ organiza os diferentes valores manipulados por um programa.

Em termos gerais, podemos distinguir entre **tipos fundamentais**, **tipos compostos** e **tipos definidos pelo usuário ou fornecidos pela Biblioteca Padrão**. Entre os **tipos fundamentais** encontram-se `int`, `double`, `char` e `bool`. Já `std::string` é uma **classe** disponibilizada pela **Biblioteca Padrão** para **representar** e **manipular** sequências de **caracteres**.

Essa distinção é particularmente importante em **programação orientada a objetos**: uma **variável** cujo **tipo** é uma **classe** possui um **objeto** associado, que pode oferecer **dados** e **operações** por meio de seus **membros**.

2.6.1 Tipos Fundamentais

Os tipos fundamentais fornecem os elementos básicos para a representação de valores em C++. Alguns dos tipos mais utilizados em programas introdutórios são apresentados a seguir.

Tabela 2.5: Alguns tipos fundamentais da linguagem C++.

Tipo	Finalidade	Exemplo
int	Valores inteiros	int idade{25};
double	Números de ponto flutuante	double preco{19.90};
char	Um único caractere	char inicial{'A'};
bool	Valores lógico-boleanos	bool ativo{true};

Esses tipos fazem parte da própria linguagem e podem ser utilizados sem a inclusão de uma biblioteca específica. Por exemplo:

```
1 int idade{25};  
2 double altura{1.75};  
3 char inicial{'M'};  
4 bool ativo{true};
```

Cada variável possui um tipo conhecido em tempo de compilação. O tipo determina, entre outras características, quais valores podem ser representados e quais operações podem ser realizadas sobre o objeto.

2.6.2 A Classe std::string

Embora o C++ possua o tipo fundamental char, uma aplicação normalmente precisa manipular sequências compostas por vários caracteres, como nomes, endereços, mensagens e identificadores.

A Biblioteca Padrão fornece a classe std::string para essa finalidade. Ela representa uma sequência de caracteres e oferece diversas operações para sua construção, consulta e modificação.

Para utilizar std::string, devemos incluir o cabeçalho <string>:

```
1 #include <string>
```

O nome `string` pertence ao *namespace* `std`. Por isso, quando não utilizamos uma declaração `using`, devemos escrever:

```
1 std::string nome{"Maria"};
```

A expressão anterior declara um **objeto** denominado `nome`, cujo **tipo** é `std::string`, e o **inicializa** com a **sequência de caracteres** `"Maria"`.

NOTA

`std::string` não é um **tipo fundamental** como `int` ou `double`. Trata-se de uma **classe** da **Biblioteca Padrão** do C++. Consequentemente, objetos desse tipo possuem **operações** e **funções-membro** que podem ser utilizadas pelo programa.

2.6.3 Operações Básicas com `std::string`

A **classe** `std::string` disponibiliza **operações** que simplificam consideravelmente a **manipulação de texto**. Uma das operações mais básicas é a atribuição de uma **nova sequência**:

```
1 std::string nome{"Maria"};
2
3 nome = "Joao";
```

Também é possível consultar o **número de caracteres** armazenados utilizando a função-membro `size()`:

```
1 std::string nome{"Maria"};
2
3 std::cout << "Quantidade de caracteres: "
4           << nome.size()
5           << '\n';
```

O resultado será:

Quantidade de caracteres: 5

A função `size()` retorna a quantidade de **elementos** armazenados na sequência. O resultado não deve ser confundido com o **tamanho físico** do objeto em **memória**.

2.6.4 Acesso aos **Caracteres** de uma `std::string`

Uma `std::string` é uma sequência **ordenada** de caracteres. Cada caractere possui uma posição, denominada **índice**. Assim como ocorre com outras **sequências** em C++, a **indexação** começa em zero.

Considere:

```
1 std::string nome{"Maria"};
2 std::cout << nome[0] << '\n';
3 std::cout << nome[1] << '\n';
4 std::cout << nome[4] << '\n';
```

A saída será:

M
a
a

Portanto, para a sequência "Maria", temos:

Tabela 2.6: **Indexação** dos caracteres de uma `std::string`.

Índice	0	1	2	3	4
Caractere	M	a	r	i	a

ⓘ Erro Comum de Programação 2.9

Tentar acessar uma posição **inexistente** da sequência. Para uma `std::string` com `n` caracteres, os **índices** válidos para acesso por meio de operador `[]` são de 0 até `n - 1`.

2.6.5 Concatenação de Strings

A classe `std::string` também permite combinar sequências de caracteres. O operador `+` pode ser utilizado para produzir uma nova string resultante da **concatenação** de seus operandos.

```
1 std::string nome{"Maria"};
2 std::string sobrenome{"Silva"};
3
4 std::string nomeCompleto = nome + " " + sobrenome;
5
6 std::cout << nomeCompleto << '\n';
```

Nesse exemplo, o operador `+` não realiza uma soma numérica. Seu comportamento depende dos **tipos** dos operandos. Quando aplicado a **objetos** `std::string` e sequências compatíveis, ele produz uma nova sequência resultante da **concatenação**.

NOTA

O significado de um **operador** em C++ depende dos **tipos** envolvidos na expressão. Essa característica está relacionada à **sobrecarga de operadores**, conceito que será aprofundado posteriormente no estudo da **programação orientada a objetos**.

2.6.6 `std::string` e `std::cout`

Para exibir uma string no terminal, podemos utilizar o objeto `std::cout`, disponibilizado pelo **cabeçalho** `<iostream>`. Quando um programa utiliza simultaneamente `std::string` e `std::cout`, os dois cabeçalhos devem ser incluídos:

```
1 #include <iostream>
2 #include <string>
3
4 int main()
5 {
6     std::string nome{"Maria"};
7 }
```

```
8     std::cout << "Nome: " << nome << '\n';
9
10    return 0;
11 }
```

O cabeçalho `<string>` disponibiliza a declaração da classe `std::string`, enquanto `<iostream>` disponibiliza os recursos necessários para a entrada e saída por fluxo utilizados nesse exemplo.

Boa Prática de Programação 2.9: Inclua os cabeçalhos necessários

Inclua **explicitamente** os cabeçalhos necessários para utilizar os recursos empregados pelo **arquivo-fonte**. Se o programa utiliza `std::string` e `std::cout`, inclua `<string>` e `<iostream>`, respectivamente. Não dependa da inclusão **indireta** de um cabeçalho por outra biblioteca.

2.6.7 Exemplo Integrado: Cadastro de um Produto

Podemos agora aplicar esses conceitos à classe `Produto` desenvolvida nas seções anteriores.

Código 2.16: `Produto.h`

```
1 #pragma once
2
3 #include <string>
4
5 class Produto
6 {
7     private:
8         std::string nome;
9         double preco;
10
11     public:
12         Produto(const std::string& nome, double preco);
13
14         std::string getNome() const;
15         double getPreco() const;
```

```
16
17     void setNome(const std::string& nome);
18     void setPreco(double preco);
19 };
```

A classe utiliza dois tipos diferentes de dados: `std::string` para representar o nome do produto e `double` para representar seu preço.

Sua implementação pode ser organizada no arquivo `Produto.cpp`:

Código 2.17: Produto.cpp

```
1  #include "Produto.h"
2
3  Produto::Produto(const std::string& nome, double preco)
4      : nome{nome}, preco{preco}
5  {
6  }
7
8  std::string Produto::getNome() const
9  {
10     return nome;
11 }
12
13 double Produto::getPreco() const
14 {
15     return preco;
16 }
17
18 void Produto::setNome(const std::string& nome)
19 {
20     this->nome = nome;
21 }
22
23 void Produto::setPreco(double preco)
24 {
25     if (preco >= 0.0)
26     {
27         this->preco = preco;
28     }
```

29 }

O programa cliente pode então utilizar a classe normalmente:

Código 2.18: main.cpp

```
1 #include <iostream>
2 #include "Produto.h"
3
4 int main()
5 {
6     Produto produto{"Teclado", 150.00};
7
8     std::cout << "Produto: "
9               << produto.getNome()
10              << '\n';
11
12     std::cout << "Preço: R$ "
13               << produto.getPreco()
14              << '\n';
15
16     return 0;
17 }
```

Esse exemplo reúne vários conceitos estudados até aqui: classes, objetos, atributos privados, construtores, funções-membro, encapsulamento, `std::string`, `std::iostream`, separação entre interface e implementação e compilação de múltiplos arquivos.

2.6.8 Exercício Prático: Cadastro de um Livro

Lista de Exercícios

Implemente uma classe denominada `Livro`, organizando o programa em arquivos separados.

A classe deverá possuir os seguintes atributos privados:

- `std::string titulo;`
- `std::string autor;`
- `int anoPublicacao.`

Implemente:

- (a) um construtor para inicializar os três atributos;
- (b) uma função-membro `getTitulo()`;
- (c) uma função-membro `getAutor()`;
- (d) uma função-membro `getAnoPublicacao()`;
- (e) uma função-membro `setAnoPublicacao()` que rejeite anos menores ou iguais a zero.

Organize a implementação nos arquivos:

```
Livro.h  
Livro.cpp  
main.cpp
```

No arquivo `main.cpp`, crie pelo menos dois objetos `Livro` e apresente seus dados utilizando `std::cout`.

Compile o projeto utilizando:

```
g++ main.cpp Livro.cpp -o livros
```

e execute o programa.

O objetivo do exercício é consolidar a utilização de `std::string` dentro de uma classe e reforçar a separação entre interface, implementação e código cliente.

👍 Boa Prática de Programação 2.10: Evite C-style strings sem necessidade

Para representar texto em código C++ moderno, prefira `std::string` quando a semântica desejada for uma sequência de caracteres gerenciada pelo programa. O uso de arrays de `char` e ponteiros para C-style strings será abordado posteriormente, quando forem necessários para compreender interoperabilidade e aspectos de baixo nível da linguagem.

Lista de Exercícios

Estudo de Caso Comparativo: Expressando Paradigmas em C++

A linguagem C++ é reconhecida por sua natureza multiparadigma. Uma mesma tarefa de processamento de dados pode ser resolvida utilizando diferentes abordagens mentais, modelos de execução e garantias de projeto.

Considere o código C++ abaixo, que resolve o mesmo problema — processar uma lista de notas, filtrar os alunos aprovados e calcular a média — utilizando três paradigmas distintos: Imperativo/Estruturado, Orientado a Objetos e Funcional/Declarativo.

Código 2.19: compare.cpp

```
1  #include <iostream>
2  #include <vector>
3  #include <numeric>
4  #include <algorithm>
5  #include <ranges>
6
7  // =====
8  // PARADIGMA 1: IMPERATIVO / ESTRUTURADO
9  // Foco: Passos instrucionais, mutabilidade explícita, controle de fluxo manual
10 // =====
11 void processarImperativo(const std::vector<double>& notas) {
12     double soma = 0.0;
13     int quantidadeAprovados = 0;
14
15     // Iteracao manual com alteracao de estado (efeitos colaterais locais)
16     for (size_t i = 0; i < notas.size(); ++i) {
17         if (notas[i] >= 7.0) {
18             soma += notas[i];
19             quantidadeAprovados++;
20         }
21     }
22
23     double media = (quantidadeAprovados > 0) ? (soma / quantidadeAprovados) :
        0.0;
```

```
24     std::cout << "[Imperativo] Aprovados: " << quantidadeAprovados
25         << " | Media: " << media << '\n';
26 }
27
28 // =====
29 // PARADIGMA 2: ORIENTADO A OBJETOS (POO)
30 // Foco: Encapsulamento de estado e comportamento, abstracao do dominio.
31 // =====
32 class AnalisadorNotas {
33 private:
34     std::vector<double> notas;
35     double notaCorte;
36
37 public:
38     explicit AnalisadorNotas(std::vector<double> n, double corte = 7.0)
39         : notas(std::move(n)), notaCorte(corte) {}
40
41     std::vector<double> obterAprovados() const {
42         std::vector<double> aprovados;
43         for (double nota : notas) {
44             if (nota >= notaCorte) {
45                 aprovados.push_back(nota);
46             }
47         }
48         return aprovados;
49     }
50
51     double calcularMedia(const std::vector<double>& conjunto) const {
52         if (conjunto.empty()) return 0.0;
53         double soma = 0.0;
54         for (double val : conjunto) soma += val;
55         return soma / conjunto.size();
56     }
57
58     void exibirRelatorio() const {
59         auto aprovados = obterAprovados();
60         double media = calcularMedia(aprovados);
61         std::cout << "[POO] Aprovados: " << aprovados.size()
62             << " | Media: " << media << '\n';
```

```

63     }
64 };
65
66 // =====
67 // PARADIGMA 3: FUNCIONAL / DECLARATIVO (C++20 Ranges)
68 // Foco: Imutabilidade, composicao de funcoes, "o que fazer" em vez de "como
    fazer".
69 // =====
70 void processarFuncional(const std::vector<double>& notas) {
71     // Predicado para filtragem
72     auto ehAprovado = [](double n) { return n >= 7.0; };
73
74     // Composicao declarativa via pipelines (C++20)
75     auto aprovadosView = notas | std::views::filter(ehAprovado);
76
77     // Avaliacao e reducao sem mutabilidade explicita em lacos
78     std::vector<double> aprovados(aprovadosView.begin(), aprovadosView.end());
79
80     double soma = std::accumulate(aprovados.begin(), aprovados.end(), 0.0);
81     double media = !aprovados.empty() ? (soma / aprovados.size()) : 0.0;
82
83     std::cout << "[Funcional] Aprovados: " << aprovados.size()
84               << " | Media: " << media << '\n';
85 }
86
87 int main() {
88     const std::vector<double> turmas = {8.5, 4.2, 7.0, 9.1, 5.5, 6.8, 10.0};
89
90     processarImperativo(turmas);
91
92     AnalisadorNotas analisador(turmas);
93     analisador.exibirRelatorio();
94
95     processarFuncional(turmas);
96
97     return 0;
98 }

```

Com base no código acima e nos conceitos estudados ao longo das Unidade 01 e 02, responda às seguintes questões analíticas:

- (a) **Gerenciamento de Estado e Mutabilidade:** compare a variável `soma` na função `processarImperativo` com a abordagem de acumulação utilizada na função `processarFuncional`. Explique como a minimização ou ausência de mutabilidade de variáveis locais contribui para a prevenção de efeitos colaterais e facilidade de paralelização do código.
- **Comparação:** na função `processarImperativo`, a variável `soma` é continuamente modificada a cada iteração do laço (`soma += ...`), caracterizando um *estado mutável explicitado*. Na abordagem `processarFuncional`, o cálculo utiliza a função pura `std::accumulate` sobre uma coleção imutável, produzindo o resultado em um único passo de avaliação sem reatribuições explícitas.
 - **Efeitos Colaterais e Paralelização:** a eliminação ou minimização de variáveis mutáveis previne reentrâncias indevidas e condições de corrida (*race conditions / data races*). Como as variáveis não mudam de valor após sua criação, múltiplas *threads* podem ler os mesmos dados em paralelo sem a necessidade de mecanismos complexos de travamento (*mutexes* ou *locks*), tornando o código *thread-safe* por natureza.
- (b) **Encapsulamento e Ocultação de Informação (POO):** na classe `AnalizadorNotas`, analise a separação entre os dados membros (`notas`, `notaCorte`) e seus métodos. Qual é a vantagem arquitetural dessa abordagem em relação às funções livres do paradigma imperativo no que tange à manutenção e reusabilidade do código em projetos de grande porte?
- **Vantagem Arquitetural:** ao agrupar os dados (`notas`, `notaCorte`) e os comportamentos (`obterAprovados`, `calcularMedia`) na classe `AnalizadorNotas`, protege-se o estado interno contra modificações inadequadas vindas do exterior da classe (garantia de invariante de classe).
 - **Manutenção e Reusabilidade:** diferente das funções livres imperativas — que dependem da passagem constante de parâmetros soltos e mantêm o estado disperso —, o encapsulamento permite alterar a estrutura interna

de dados (ex.: trocar `std::vector` por `std::list`) sem quebrar a interface pública utilizada pelo restante do sistema. Em projetos de grande porte, isso reduz o acoplamento e facilita a criação de instâncias reutilizáveis com diferentes configurações de estado.

- (c) **Expressividade e Nível de Abstração:** no trecho declarativo/funcional, fez-se uso de *Pipelines* (`std::views::filter`) introduzidos no C++20. Explique a diferença entre descrever “*como iterar*” (visão imperativa) e “*o que transformar/filtrar*” (visão funcional), destacando o impacto dessa diferença na legibilidade do código.
- **Manutenção e Reusabilidade: “Como iterar” vs. “O que transformar/filtrar”:** o modelo imperativo foca no *mecanismo de execução*: gerenciar explicitamente os índices, verificar limites do vetor e ramificar o fluxo com instruções de controle (`for`, `if`). O modelo funcional/declarativo via *C++20 Ranges* foca na *intenção do negócio*: expressa diretamente a regra de filtrar os dados segundo um predicado (`std::views::filter(ehAprovado)`), delegando a infraestrutura da iteração para a linguagem.
 - **Manutenção e Reusabilidade: Impacto na Legibilidade:** a abordagem declarativa elimina o código utilitário de controle (*boilerplate*), aproximando a sintaxe do problema do domínio conceitual. A leitura se torna linear (da esquerda para a direita via operador `|`), facilitando a auditoria do código, a prevenção de erros comuns de limites (*off-by-one errors*) e aumentando a manutenibilidade.
- (d) **Matriz de Trade-offs:** preencha o quadro comparativo abaixo sintetizando os impactos de cada paradigma segundo os critérios de engenharia de software observados no exercício:

Critério	Imperativo	Orientado a Objetos	Funcional
Curva de Aprendizado	Baixa	Média	Média / Alta
Gestão de Estado	Variáveis mutáveis e controle manual	Estado encapsulado dentro de objetos	Prevalência de imutabilidade e expressões
Foco de Abstração	Passos do algoritmo / Hardware	Entidades do domínio / Classes	Fluxo de dados / Transformações
Facilidade de Testes	Baixa (depende do estado global/local)	Média (exige instancia de objetos)	Alta (funções puras e sem efeitos colaterais)

Tabela 2.7: Quadro Comparativo de Paradigmas em C++